

실습: SLAM Toolbox를 활용한 Gazebo 시뮬레이션 매핑

사용자 정의 로봇 모델로 지도 생성·저장·확인

cchyun@gmail.com

Part 1. 실습 개요 및 시스템 구조

- `my_robot_description` 패키지의 Gazebo 시뮬레이터에서 SLAM Toolbox로 실시간 매핑 수행
 - `slam.launch.py` 작성 → Gazebo 기동 → 매핑 → 지도 저장 → 재로드 확인까지 완료
 - 로봇의 센서 데이터가 어떻게 지도로 변환되는지 이해
- 실습 7단계 흐름
 1. `slam.launch.py` 작성 및 Gazebo World 기동
 2. SLAM Toolbox 비동기 모드 런치
 3. RViz2 시각화 설정
 4. 수동 주행하며 매핑 수행
 5. 맵 품질 확인 및 미탐색 영역 재주행
 6. `map_saver_cli` 로 지도 저장
 7. 저장된 지도 재로드 및 확인

- 주요 노드 (Nodes)

- `gazebo` : 가상 환경 시뮬레이션 및 로봇 물리 엔진 구동
- `async_slam_toolbox_node` : LiDAR 데이터 + Odometry를 결합해 실시간 지도 생성하는 핵심 엔진
- `teleop_twist_keyboard` : 키보드 입력을 로봇의 이동 명령으로 변환
- `rviz2` : 매핑 과정을 눈으로 확인하기 위한 시각화 도구

- 핵심 토픽 (Topics)

- `/scan` : LiDAR 센서가 측정한 장애물 거리 정보
- `/odom` : 바퀴 회전수 기반 상대적 위치 정보
- `/map` : 생성된 격자 지도 (Occupancy Grid)
- `/cmd_vel` : 로봇의 선속도 및 각속도 제어 명령

- TF 체인은 로봇의 각 파트 간 좌표 변환 관계를 정의

- 체인 구조

```
map → odom → base_link → base_scan
```

- SLAM Toolbox의 역할

- `map` 프레임 ↔ `odom` 프레임 사이의 변환(오차 보정값)을 계산하여 발행
- 바퀴 미끄러짐 등으로 누적되는 드리프트(drift) 오차를 실시간 보정

- 왜 중요한가?

- TF가 올바르게 연결되어야 RViz에서 로봇 위치와 지도가 정확히 일치
- `use_sim_time:=True` 미설정 시 TF 타임스탬프 불일치로 오류 발생

Part 2. 단계별 실습 구현

사전 준비 — slam.launch.py 작성

- `my_robot_description` 패키지에 `slam.launch.py` 생성
 - 10.03에서 작성한 `gazebo.launch.py`의 `world_file` 경로를 아래와 같이 변경

```
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import ExecuteProcess, DeclareLaunchArgument
from launch.substitutions import Command, LaunchConfiguration, PathJoinSubstitution
from launch_ros.actions import Node

def generate_launch_description():
    declare_world_arg = DeclareLaunchArgument(
        'world',
        default_value='room_world.world',
        description='Gazebo world file name'
    )

    pkg_dir = get_package_share_directory('my_robot_description')
    xacro_file = os.path.join(pkg_dir, 'urdf', 'turtlebot.xacro')
    world_file = PathJoinSubstitution([pkg_dir, 'worlds', LaunchConfiguration('world')])
    rviz_file = os.path.join(pkg_dir, 'rviz', 'slam.rviz')

    robot_description = Command(['xacro ', xacro_file])
    rviz_args = ['-d', rviz_file] if os.path.exists(rviz_file) else []
```

```
return LaunchDescription([
    declare_world_arg,
    ExecuteProcess(
        cmd=['gazebo', '--verbose', world_file,
            '-s', 'libgazebo_ros_init.so',
            '-s', 'libgazebo_ros_factory.so'],
        output='screen'
    ),
    Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        parameters=[{
            'robot_description': robot_description,
            'use_sim_time': True,
        }]
    ),
    Node(
        package='gazebo_ros',
        executable='spawn_entity.py',
        arguments=['-topic', 'robot_description',
            '-entity', 'turtlebot', '-z', '0.3'],
        output='screen'
    ),
    Node(
        package='rviz2',
        executable='rviz2',
        arguments=rviz_args,
    ),
])
```


- 패키지 빌드 및 환경 설정

```
cd ~/Workspace/ros2-ws  
colcon build --packages-select my_robot_description  
source install/setup.bash
```

- 주의 사항

- 매번 새 터미널을 열 때마다 `source install/setup.bash` 를 수행해야 함
- `room_world.world` 파일이 `my_robot_description/worlds/` 에 위치하는지 확인

- 가상 시뮬레이션 환경을 실행

```
source install/setup.bash
ros2 launch my_robot_description slam.launch.py
```

- 예상 화면

- 5m×5m 공간에 박스·원통 장애물이 배치된 Gazebo 시뮬레이터 창이 뜸
- 중앙에 로봇이 안정적으로 나타남

- 확인 포인트

- 로봇이 물리 엔진에 의해 바닥에 안정적으로 서 있는지 확인

2단계 — slam_param.yaml 작성

```
slam_toolbox:
  ros__parameters:
    odom_frame: odom          # 오도메트리 좌표계 프레임 (odom → base_footprint 변환 제공)
    map_frame: map            # 맵 좌표계 프레임 (map → odom 변환을 publish)
    base_frame: base_footprint # 로봇 베이스 프레임 (센서/바퀴의 기준 좌표계)
    scan_topic: /scan         # 구독할 LiDAR 스캔 토픽 이름
    mode: mapping             # SLAM 모드: mapping (맵 생성) 또는 localization (위치 추정)
    transform_publish_period: 0.02 # TF(map→odom) 발행 주기 (초), 50Hz
    resolution: 0.05          # Occupancy Grid 맵 해상도 (m/픽셀), 5cm
    minimum_travel_distance: 0.05 # 맵 업데이트 최소 직진 이동 거리 (m)
    minimum_travel_heading: 0.05  # 맵 업데이트 최소 회전 각도 (rad)
    use_scan_matching: true      # 스캔 매칭 활성화 (Odometry drift 보정)
    do_loop_closing: true        # 루프 클로저 활성화 (이전 경로 재방문 시 보정)
```

- 사전 설치 확인

```
sudo apt update
sudo apt install -y ros-humble-slam-toolbox
sudo apt install -y ros-humble-navigation2 ros-humble-nav2-bringup
```

- 새 터미널에서 SLAM Toolbox 실행

```
source ~/Workspace/ros2-ws/install/setup.bash
ros2 launch slam_toolbox online_async_launch.py \
use_sim_time:=True \
slam_params_file:=src/my_robot_description/config/slam_param.yaml
```

- 핵심 옵션: `use_sim_time:=True`

- Gazebo의 시뮬레이션 시간과 SLAM Toolbox의 타임스탬프를 동기화하는 필수 옵션
- 누락 시 로봇이 떨리거나 TF 오류가 발생함

- 비동기(Asynchronous) 모드를 사용하는 이유

- 저사양 환경에서도 최신 스캔 데이터를 우선 처리하여 실시간 주행 가능
- 동기 모드 대비 CPU 부하를 낮춰 시뮬레이션 성능 유지

- RViz 설정 방법 (순서대로 진행)

- 좌측 패널 `Fixed Frame` 을 `map` 으로 설정
- 하단 `Add` 버튼 클릭 → `By topic` 탭 선택
- 아래 항목 추가:

추가 항목	Topic	주의 사항
Map	<code>/map</code>	Durability: <code>Transient Local</code> 로 반드시 설정
LaserScan	<code>/scan</code>	기본값으로 추가
TF	—	기본값으로 추가

- `/map` QoS를 `Transient Local` 로 설정하면, RViz2를 나중에 실행하더라도 지도가 즉시 표시됨

4단계 — 수동 주행하며 매핑 수행

- 로봇을 움직여 지도를 그림

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

- 권장 주행 패턴

- 느린 속도 유지

- 약 0.1 m/s 정도로 천천히 이동
- 빠른 이동 시 LiDAR 매칭 실패 → 지도 왜곡·겹침 발생

- 루프 주행 (Loop Closure)

- 이미 방문한 장소를 다시 방문
- SLAM Toolbox가 **Loop Closure**를 감지하여 누적 오차를 자동 보정
- 시작 지점으로 돌아오는 경로를 의식적으로 계획할 것

- 회전 시 주의

- 급격한 회전은 오도메트리 오차를 키움
- 천천히 부드럽게 회전할 것

- 품질 평가 기준

- 좋은 지도의 특징

- 벽면이 한 줄로 선명하게 표현됨
- 유령 현상(Ghosting) 없음 — 같은 벽이 두 겹으로 보이는 현상
- 직선 벽이 지도에서도 완벽한 직선으로 표현됨

- 나쁜 지도의 징후와 대응

- 벽이 흐리거나 두 겹으로 보임 → 속도를 줄이고, 이미 지나간 구역을 다시 방문하여 Loop Closure 유도
- 회색(Unknown) 영역이 많음 → 해당 구역을 직접 방문하여 탐색

- 미탐색 영역 처리

- RViz에서 회색(Unknown) 으로 표시된 부분 확인
- 로봇으로 해당 구역을 직접 방문 → 흰색(Free space) 으로 전환
- 회색 영역이 남아있으면 Nav2 경로 계획에서 진입 불가 구역으로 처리됨

6단계 — map_saver_cli로 지도 저장

- 완성된 지도를 파일로 영구 저장

```
ros2 run nav2_map_server map_saver_cli -f ./room_map
```

- 생성 결과 파일

파일	설명
<code>./room_map.pgm</code>	격자 지도 이미지 파일 (흑백 픽셀 이미지)
<code>./room_map.yaml</code>	지도 메타데이터 (해상도, 원점, 임계값 등)

- 저장 실패 시 확인 사항

- SLAM Toolbox가 켜져 있는지 확인
- `/map` 토픽이 발행 중인지 확인: `ros2 topic echo /map --once`
- Timeout 발생 시 명령어를 다시 실행하거나 `--timeout 10.0` 옵션 추가

라이프사이클 매니저 (Lifecycle Manager)

- ROS2의 Lifecycle 노드는 "자동으로 켜지지 않음"
 - 일반 노드: `ros2 run` 하자마자 토픽 발행/구독 시작
 - Lifecycle 노드: 실행필도 **Inactive** 상태 → 토픽 발행 안 함
 - Lifecycle Manager가 명령을 날려야 **Active** 상태로 전환됨

- Lifecycle 상태 흐름

Unconfigured → Inactive → Active

↑ ↑
생성 직후 run했지만 ← Manager가 "configure" + "activate" 명령 전송
 아직 대기

- Nav2의 map_server는 Lifecycle 노드
 - `ros2 run nav2_map_server map_server ...` 만 실행하면
 - 남부 YAML을 읽었지만 **토픽 발행은 하지 않음**
 - `nav2_lifecycle_manager` 가 `autostart:=True` 로
자동 활성화해야 `/map` 토픽이 실제로 발행됨

7단계 — 저장된 지도 재로드 및 확인

- 저장된 지도를 다시 불러와 정확성을 검증

```
# 터미널 1: map → odom TF를 임시로 발행
ros2 run tf2_ros static_transform_publisher 0 0 0 0 0 0 map odom

# 터미널 2: map_server 실행 (SLAM Toolbox 종료 후)
ros2 run nav2_map_server map_server \
  --ros-args -p yaml_filename:=./room_map.yaml

# 터미널 3: 라이프사이클 매니저로 map_server 활성화
ros2 run nav2_lifecycle_manager lifecycle_manager \
  --ros-args -p node_names:='["map_server"]' -p autostart:=True
```

- 확인 방법

- RViz2 실행 후 Fixed Frame 을 map 으로 설정
- /map 토픽 추가 → Durability Policy 를 Transient Local 로 변경
- 저장된 지도가 안정적으로 표시되면 재로드 성공

8단계 — launch 파일에 SLAM Toolbox 통합

- 별도 터미널 실행 대신 하나의 launch 파일로 통합

```
# 기존 slam.launch.py에 다음 내용 추가
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource

# SLAM Toolbox 추가
slam_launch = IncludeLaunchDescription(
    PythonLaunchDescriptionSource([
        os.path.join(
            get_package_share_directory('slam_toolbox'),
            'launch',
            'online_async_launch.py'
        )
    ]),
    launch_arguments={'use_sim_time': 'True'}.items()
)
```

- 통합 후 실행

```
cd ~/Workspace/ros2-ws  
colcon build --packages-select my_robot_description  
source install/setup.bash  
ros2 launch my_robot_description slam.launch.py
```

Part 3. SLAM Toolbox 심화

- Pose Graph란?

- SLAM 내부의 시계열 그래프 구조
- **Pose Node**: 특정 시점의 로봇 자세 (x, y, θ) + 타임스탬프
- **Laser Scan**: 그 시점의 LiDAR 원시 데이터 `ranges[]`
- **Odometry Edge**: 연속 노드 간 이동량 (휠 엔코더 기반)
- **Loop Closure Edge**: 과거 위치 재방문 시 연결 (스캔 매칭)

시간 →

`[pose_0, scan_0] -odom→ [pose_1, scan_1] -odom→ [pose_2, scan_2] -odom→ ...`

↑-----↓
(loop closure: pose_0 ↔ pose_2)

직렬화(Serialize)와 Pose Graph

- 매핑 상태 전체(Pose Graph)를 저장·복원

저장 (직렬화):

```
ros2 launch my_robot_description slam.launch.py world:=slam_world.world

ros2 service call /slam_toolbox/serialize_map \
  slam_toolbox/srv/SerializePoseGraph \
  "{filename: './slam_map_serial'}"
```

생성 파일:

- `slam_map_serial.data`
- `slam_map_serial.posegraph`

파일	저장 내용	복원 후 가능한 것
<code>.pgm</code> + <code>.yaml</code>	2D 점유 그리드 이미지	지도만 읽기 전용으로 불러오기
<code>.data</code> + <code>.posegraph</code>	Pose Graph 전체 (노드+엣지+원시 스캔)	이어서 매핑(Lifelong Mapping)

- 이어서 매핑(Lifelong Mapping) 사용 시나리오
 - 분할 매핑: 큰 공간을 여러 세션에 나눠 탐색 (충전/이동 중간 저장)
 - 동적 환경: 가구 배치 변경, 계절 변화 등 시간에 따른 지도 갱신
 - 장시간 운영: 배달/청소 로봇 24시간 순환 운영
- 복원 (Deserialize):

```
ros2 launch my_robot_description slam.launch.py world:=slam_world.world

ros2 service call /slam_toolbox/deserialize_map \
  slam_toolbox/srv/DeserializePoseGraph \
  "{filename: '/home/cchyun/Workspace/ros2_ws/slam_map_serial', match_type: 1}"
```

match_type	의미
0	현재 위치를 Pose Graph 처음 위치로 간주
1	현재 위치를 Pose Graph 마지막 위치로 간주 → 이어서 매핑

Part 4. 트러블슈팅 및 심화 미션

- 공통 디버깅 명령

```
ros2 topic list           # 현재 발행 중인 토픽 목록
ros2 topic hz /scan       # LiDAR 데이터 수신 주기 확인
ros2 topic echo /scan --once # LiDAR 데이터 유무 확인
ros2 node list            # 현재 동작 중인 노드 확인
```

- 주행 데이터를 녹화하여 오프라인에서 SLAM을 재현 (1/2)
 - 1단계: 주행 중 데이터 녹화

```
ros2 launch my_robot_description slam.launch.py  
ros2 bag record /scan /tf /tf_static
```



- 주행 데이터를 녹화하여 오프라인에서 SLAM을 재현 (2/2)

- 2단계: Gazebo 종료 후 오프라인 매핑 재현

```
# 녹화된 bag 파일 재생
ros2 bag play <녹화파일_경로> --clock

# 동시에 SLAM Toolbox 실행 (별도 터미널)
ros2 launch slam_toolbox online_async_launch.py use_sim_time:=True
```

- 필수 옵션 설명

- `ros2 bag play --clock` → rosbag 데이터의 시간을 `/clock` 토픽으로 발행. `use_sim_time:=True` 인 노드들이 이 시간을 따라가도록 함
- `rviz2 --ros-args -p use_sim_time:=True` → RViz2도 시뮬레이션 시간을 사용하도록 설정

- 학습 효과

- 동일한 데이터로 여러 파라미터 설정을 반복 테스트 가능
- 하드웨어 없이도 알고리즘 동작 방식을 심층 분석